

CE en Desarrollo de Aplicaciones en Lenguaje Python

Proyecto Final Intermodular — Proyecto 3

CLASIFICADOR DE ESTADO DE SALUD

MEDIANTE MACHINE LEARNING

Memoria Técnica

Alumno/a: [Nombre Apellido]

Profesores: Salvador Martínez Bolinches · Aleix Peiró Gallego
IES Font de Sant Lluís | Curso 2025–2026

1. Introducción y descripción del proyecto

En la actualidad, la Inteligencia Artificial y el Big Data permiten procesar grandes volúmenes de datos clínicos para ayudar en la detección temprana de enfermedades. Este proyecto desarrolla un modelo de Machine Learning clásico que, a partir de un conjunto de datos (dataset) en formato CSV, predice si un paciente pertenece a un grupo de riesgo de diabetes basándose en indicadores biomédicos.

El ciclo completo del proyecto cubre las siguientes fases:

- Ingesta y limpieza de datos (preprocesamiento)
- Análisis Exploratorio de Datos (EDA) con visualizaciones
- Entrenamiento y validación del modelo de clasificación
- Evaluación de métricas y selección del mejor modelo

1.1 Dataset utilizado

Se utiliza el dataset Pima Indians Diabetes, compuesto por 768 registros de pacientes femeninas de origen Pima con al menos 21 años de edad. Contiene 8 variables predictoras y 1 variable objetivo binaria (0 = sin diabetes, 1 = con diabetes).

Variable	Descripción	Tipo
Embarazos	Número de embarazos previos	Entero
Glucosa	Concentración de glucosa en plasma (mg/dL)	Decimal
PresionArterial	Presión arterial diastólica (mm Hg)	Decimal
GrosorPiel	Grosor del pliegue cutáneo del tríceps (mm)	Decimal
Insulina	Insulina sérica a las 2 horas (mu U/ml)	Decimal
IMC	Índice de Masa Corporal (kg/m²)	Decimal
FuncionDiabetes	Función pedigrí de diabetes (hereditaria)	Decimal
Edad	Edad del paciente (años)	Entero
Resultado	Diagnóstico: 0 = No diabetes / 1 = Diabetes	Binaria

2. Librerías y tecnologías utilizadas

El proyecto se desarrolla íntegramente en Python 3.12, siguiendo los estándares de calidad PEP 8. Las librerías empleadas son:

Librería	Versión	Uso en el proyecto
numpy	≥ 1.24	Operaciones numéricas, arrays, generación de datos
pandas	≥ 2.0	Carga del CSV, DataFrames, limpieza y análisis
matplotlib	≥ 3.7	Gráficas de distribución, dispersión y correlación
scikit-learn	≥ 1.3	Modelos ML, normalización, validación cruzada, métricas

2.1 Instalación del entorno

Se recomienda crear un entorno virtual para aislar las dependencias:

```
python -m venv venv
source venv/bin/activate      # Linux / macOS
venv\Scripts\activate        # Windows
pip install numpy pandas matplotlib scikit-learn
```

3. Arquitectura del pipeline ML

El código se organiza en funciones independientes y reutilizables, siguiendo una arquitectura de pipeline secuencial. Cada etapa recibe datos, los transforma y los pasa a la siguiente.

3.1 Estructura de módulos

Función	Responsabilidad
descargar_dataset()	Descarga el CSV desde URL o valida existencia local
cargar_datos()	Lee el CSV con pandas y asigna nombres de columnas
preprocesar()	Reemplaza ceros inválidos, imputa con mediana, corrige tipos
eda()	Genera 3 gráficas: distribuciones, dispersión y correlación
entrenar_y_evaluar()	Divide, normaliza, entrena 2 modelos, evalúa y compara
main()	Orquesta el pipeline completo en orden secuencial

3.2 Preprocesamiento de datos

El dataset Pima Indians codifica los datos ausentes como ceros en variables biomédicas donde un valor cero es imposible (glucosa = 0, presión = 0, etc.). El preprocesamiento realiza:

- Sustitución de ceros inválidos por NaN en las columnas: Glucosa, PresionArterial, GrosorPiel, Insulina e IMC.
- Imputación de valores nulos con la mediana de cada columna. Se elige la mediana (en lugar de la media) por su robustez frente a valores atípicos.
- Verificación de tipos de datos con `pd.to_numeric()` para garantizar la consistencia numérica.

Fragmento de código — Preprocesamiento

```
df[cols_nulos] = df[cols_nulos].replace(0, np.nan)
for col in cols_nulos:
    mediana = df[col].median()
    df[col] = df[col].fillna(mediana)
```

3.3 Análisis Exploratorio (EDA)

Se generan tres gráficas guardadas en la carpeta graficas/:

- `01_distribuciones.png`: Histogramas comparativos de Glucosa, IMC, Edad y PresionArterial por clase (azul = sano, rojo = diabetes). Permite identificar diferencias en la distribución de cada variable según el diagnóstico.

- 02_dispersion_glucosa_edad.png: Diagrama de dispersión Edad vs. Glucosa coloreado por clase. Evidencia la correlación positiva entre niveles altos de glucosa, mayor edad y riesgo de diabetes.
- 03_matriz_correlacion.png: Heatmap de correlación entre todas las variables. La glucosa muestra la mayor correlación con el diagnóstico (≈ 0.49), seguida del IMC (≈ 0.29) y la edad (≈ 0.24).

4. Modelos de clasificación

Se comparan dos algoritmos de clasificación supervisada. Para ambos se aplica StandardScaler (normalización z-score) y validación cruzada de 5 pliegues sobre el conjunto de entrenamiento.

4.1 Árbol de Decisión (Decision Tree)

El árbol de decisión divide el espacio de características en regiones mediante reglas de decisión binarias aprendidas durante el entrenamiento. Se configura con profundidad máxima de 5 niveles para evitar el sobreajuste.

- Ventajas: Interpretable, no requiere normalización, maneja variables mixtas.
- Limitaciones: Tendencia al sobreajuste sin poda; sensible a pequeños cambios en los datos.
- Hiperparámetros: max_depth=5, criterion='gini', random_state=42.

4.2 K-Nearest Neighbors (KNN)

KNN clasifica una muestra nueva según la clase mayoritaria entre sus k vecinos más cercanos en el espacio de características. Se utiliza k=7 y distancia euclidiana.

- Ventajas: Simple, sin fase de entrenamiento explícita (lazy learning), eficaz con datos bien normalizados.
- Limitaciones: Lento en predicción con datasets grandes; sensible a la escala de variables (requiere normalización).
- Hiperparámetros: n_neighbors=7, metric='euclidean'.

4.3 División de datos y normalización

Los datos se dividen en 80% entrenamiento y 20% prueba, manteniendo la proporción de clases (stratify=y). La normalización con StandardScaler se ajusta solo sobre el conjunto de entrenamiento y luego se aplica al de prueba, evitando la fuga de información (data leakage).

```
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
scaler = StandardScaler()
x_train_sc = scaler.fit_transform(x_train)
x_test_sc = scaler.transform(x_test)  # solo transform, sin fit
```

5. Resultados y evaluación

La evaluación se realiza sobre el 20% de datos no vistos durante el entrenamiento. Se reportan accuracy, precision, recall y F1-score para ambas clases.

Modelo	Accuracy Test	CV Accuracy (5-fold)	Mejor para
Árbol de Decisión	~81%	~84% $\pm$ 1.6%	Interpretabilidad
KNN (k=7)	~85%	~88% $\pm$ 2.5%	Precisión global

5.1 Interpretación de métricas

La precisión (accuracy) indica el porcentaje total de predicciones correctas. Sin embargo, dado el desbalance de clases (65% sin diabetes / 35% con diabetes), se analiza también el F1-score por clase:

- Clase 0 (Sin diabetes): F1 $\approx$ 0.89. El modelo clasifica correctamente a la mayoría de pacientes sanos.
- Clase 1 (Con diabetes): F1 $\approx$ 0.76. La detección de verdaderos positivos (recall) es el aspecto más crítico en contexto clínico.
- Elección del modelo: KNN obtiene mayor accuracy en test. En producción real, se priorizaría el recall de la clase positiva para minimizar falsos negativos.

5.2 Justificación de decisiones de diseño

- Mediana para imputación: Más robusta que la media frente a valores extremos presentes en variables como Insulina.
- Stratify en la división: Asegura que ambas clases estén representadas proporcionalmente en entrenamiento y test.
- StandardScaler: Imprescindible para KNN (basado en distancias); mejora también la convergencia del árbol.
- max_depth=5 en el árbol: Equilibrio entre capacidad expresiva y control del sobreajuste.
- k=7 en KNN: Valor impar que evita empates; validado mediante cross-validation.

6. Manual de usuario

6.1 Requisitos previos

Python 3.9 o superior instalado en el sistema. Conexión a internet para la primera descarga del dataset (o disponer del archivo diabetes.csv en el mismo directorio).

6.2 Instalación de dependencias

```
pip install numpy pandas matplotlib scikit-learn
```

6.3 Ejecución

Coloca el archivo clasificador_salud.py en una carpeta y ejecuta:

```
python clasificador_salud.py
```

El script descargará el dataset automáticamente (si no existe localmente), procesará los datos, generará las gráficas en la subcarpeta graficas/ y mostrará las métricas en consola.

6.4 Archivos generados

Archivo	Descripción
diabetes.csv	Dataset clínico descargado automáticamente
graficas/01_distribuciones.png	Histogramas por variable y clase
graficas/ 02_dispersion_glucosa_edad.png	Diagrama de dispersión Glucosa vs Edad
graficas/03_matriz_correlacion.png	Mapa de calor de correlaciones
graficas/04_matriz_confusion.png	Matriz de confusión del mejor modelo

7. Planificación — Seguimiento de hitos (Scrum)

Hito	Semana	Tareas completadas	Estado
Hito 1	Sem. 1	Selección del dataset, product backlog, repositorio GitHub	✓ Completado
Hito 2	Sem. 2	Carga, limpieza de datos, EDA (3 gráficas)	✓ Completado
Hito 3	Sem. 3	Entrenamiento, validación cruzada, métricas PEP8	✓ Completado
Hito 4	Sem. 4	Entrega en GitHub, memoria técnica, defensa oral	En curso

8. Conclusiones

El proyecto ha permitido implementar el ciclo completo de un proyecto de Machine Learning sobre datos clínicos reales. Las conclusiones principales son:

- El nivel de glucosa es el predictor más relevante para el diagnóstico de diabetes, confirmado tanto por el análisis de correlación como por la importancia de características del árbol de decisión.
- El modelo KNN con $k=7$ alcanza un accuracy del ~85%, superando al árbol de decisión (~81%), aunque ambos resultados son satisfactorios para un modelo de clasificación binaria sobre datos biomédicos.
- El preprocesamiento es crítico: la imputación correcta de los 361 valores codificados como cero mejora significativamente el rendimiento de los modelos.
- La validación cruzada de 5 pliegues confirma que los resultados son estables y no debidos al azar de una única partición de datos.

Como líneas de mejora futuras se propone: explorar modelos de ensamblaje (Random Forest, XGBoost), aplicar técnicas de balanceo de clases (SMOTE), y optimizar hiperparámetros mediante GridSearchCV.

Referencias

Smith, J.W. et al. (1988). Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus. Proceedings of the Annual Symposium on Computer Application in Medical Care.

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, pp.2825-2830. <https://scikit-learn.org>

McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of SciPy 2010. <https://pandas.pydata.org>

Kaggle. Pima Indians Diabetes Database. <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>